

SQL and Analytics Interview Practice

SQL interview quality depends on the data contract as much as syntax. Define row grain, keys, timezone, ties, and missing-value meaning before choosing joins or windows. Use small hand-checkable datasets and adversarial fixtures to verify results rather than trusting a query because it executes.

For: Backend, analytics, data-engineering, and ML candidates who need correct queries under ambiguous requirements, including ties, duplicate events, missing data, and temporal boundaries.

Suggested pace: 2 weeks with a local database | Basic SELECT and GROUP BY

What you will be able to do

- Prevent double counting by reasoning about join cardinality and grain.
- Use ranking and window frames with explicit tie and ordering semantics.
- Define cohorts, retention, and funnels with valid time boundaries.
- Explain query performance using access patterns and observed plans.

Build the right mental model

Grain comes before aggregation

State what each input and output row represents. Joining two child tables can multiply records before SUM or COUNT executes. Pre-aggregate at the needed key or restructure the query; DISTINCT is not a universal repair for inflated measures.

Window functions preserve rows

ROW_NUMBER selects rows under a deterministic order, RANK leaves gaps after ties, and DENSE_RANK groups distinct rank levels without gaps. Running totals need an explicit frame when peer behavior matters. Seven rows are not automatically seven calendar days.

Time and missingness are business definitions

A NULL may mean unknown rather than zero. A day depends on timezone, and a retention cohort must have a complete observation window. A user performing all event types does not prove an ordered funnel was completed within the required session or time limit.

Study with evidence.

Write the expected result for a tiny dataset before writing SQL. Include duplicates, ties, unmatched rows, NULLs, and immature time windows. State the SQL dialect because date arithmetic and query-planning details vary across engines.

Days 1-3: Joins and grain

Practice inner and left joins, aggregation, anti-joins, and NULL behavior. Build customers, orders, and payments with deliberate duplicate and missing relationships. Explain why a predicate in WHERE can change left-join semantics.

Deliverable and check: Queries with documented output grain and fixtures that catch row multiplication.

Days 4-6: Windows and ranking

Practice top-N per group, latest-record deduplication, running totals, and gaps-and-islands. Define ties and stable secondary keys. Use outer queries or CTEs when filtering a window result.

Deliverable and check: A ranking example where exactly two rows differs from the top two distinct score levels.

Days 7-10: Temporal analytics

Define cohorts, day-based retention, ordered funnels, and rolling windows. Deduplicate at the proper entity/time grain and account for timezone, delayed events, and incomplete observation periods.

Deliverable and check: A metric specification and query that agree on numerator, denominator, ordering, and maturity.

Days 11-14: Plans and mixed problems

Inspect plans on a larger synthetic dataset, add justified indexes, and compare results before and after optimization. Solve an unseen query with a short explanation of semantics and expected cost.

Deliverable and check: A complete interview answer with assumptions, query structure, tests, and performance caveats.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Top two salary levels is not always two employees

A department has salaries 100, 100, 90, and 80. The request says 'employees in the top two salaries.' Clarify whether this means two rows or two distinct salary levels before selecting a ranking function.

A worked approach

1. If the contract means two distinct salary levels, `DENSE_RANK` partitioned by department and ordered by salary descending assigns ranks 1,1,2,3. Filtering `rank <= 2` returns three employees.
2. If the contract means exactly two employees, `ROW_NUMBER` with salary descending and a stable employee-ID tie-breaker returns two rows. The tie-breaker is a business or deterministic selection rule, not an accidental database order.
3. Place the rank calculation in a CTE or subquery and filter in the outer query. Test another department, fewer than two employees, and tied timestamps or IDs according to the schema. Explain why the two contracts legitimately produce different outputs.

Expected result

The query is correct because it matches an explicit interpretation of 'top two.' Syntax alone cannot resolve an ambiguous ranking requirement, and equal-key ordering should not be left accidental.

Verify the result

- Does the test fixture include a tie at the selection boundary?
- Is the final output order specified separately from the window order?
- Are NULL salaries allowed, and if so, is their treatment explicit?

Extend the exercise

Change the task to each customer's latest order. Add equal timestamps and a unique order-ID tie-breaker; explain why joining only to `MAX(timestamp)` can return more than one row.

Make the learning stick.

Why can NOT IN behave unexpectedly with NULL?

SQL comparisons involving NULL can evaluate to unknown. A NULL in the subquery can make unmatched comparisons fail the WHERE predicate. A carefully correlated NOT EXISTS is often clearer for an anti-join, but still verify the intended NULL contract.

Does a seven-row window represent a week?

Only if the data has exactly one row per consecutive day under the chosen calendar. Otherwise aggregate to daily grain and handle missing dates, or use an appropriate time-range method for the dialect.

Should I optimize before checking the result?

No. Establish semantics and expected output first. A faster query that changes duplicates, ties, or missing-row behavior is not an equivalent optimization. Compare results and inspect representative execution plans afterward.

Avoid these traps

- Using DISTINCT to hide an incorrect join.
- Leaving tie and timezone rules unspecified.
- Counting immature cohorts as if their observation window were complete.

Ready for the next step?

- Input and output grain are explicit.
- Fixtures cover ties, duplicates, NULLs, and missing matches.
- Temporal metrics define ordering and observation windows.
- Performance claims follow a correct query and a plausible plan.

Reliable next reads

- [PostgreSQL window functions](#)
- [PostgreSQL table expressions](#)
- [PostgreSQL EXPLAIN](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.